



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

**Кафедра квантовых информационных технологий, практической и
прикладной информатики (ККИТПиПИ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7

**по дисциплине «Проектирование предметно-ориентированных
информационных систем»**

Студент группы ИНБО-12-23, Албахтин И.В.

(подпись)

Преподаватель Борzych Н.Ю.

(подпись)

Москва 2026 г.

Ансамблевое обучение и классификация дефектов стальных пластин

Цель и задание

Цель — сравнить бэггинг и бустинг при распознавании дефектов стальных пластин. Задание требует не повторяющийся в группе набор, оба алгоритма на одинаковых данных, сравнение времени и качества, выводы, отчёт и Colab Notebook.

На едином тесте из 389 строк macro-F1 бустинга составила 0,8268, бэггинга — 0,8119. Медиана обучения — 4,715 и 0,589 с соответственно. Вывод относится к данному разбиению и среде.

1 Исходные данные

Выбран Steel Plates Faults из UCI Machine Learning Repository, авторы M. Buscema, S. Terzi и W. Tastle, 2010 год, DOI 10.24432/C5J88N, лицензия CC BY 4.0 [2]. Набор описывает семь типов дефектов; это задача многоклассовой классификации. Имена классов сохранены в написании исходного файла, включая K_Scatch.

Сведения приведены в таблице 1.

Таблица 1 – Проверенные характеристики набора

Показатель	Результат
Наблюдения	1941
Числовые входные признаки	27
Целевые классы	7
Пропуски	0
Повторы строк признаков	0
Обучающая выборка	1552
Отложенный тест	389

Архив с Faults.NNA и словарём Faults27x7_var сохранён без изменений. SHA-256 архива и таблицы проверены и записаны в results.json. Подготовка признаков описана в разделе 3.

2 Распределение классов

Сведения приведены в таблице 2.

Таблица 2 – Число наблюдений каждого класса

Класс	Всего	Train	Test
Pastry	158	126	32
Z_Scratch	190	152	38
K_Scratch	391	313	78
Stains	72	58	14
Dirtiness	55	44	11
Bumps	402	321	81
Other_Faults	673	538	135

На рисунке 1 показано неравномерное распределение классов исходного набора.

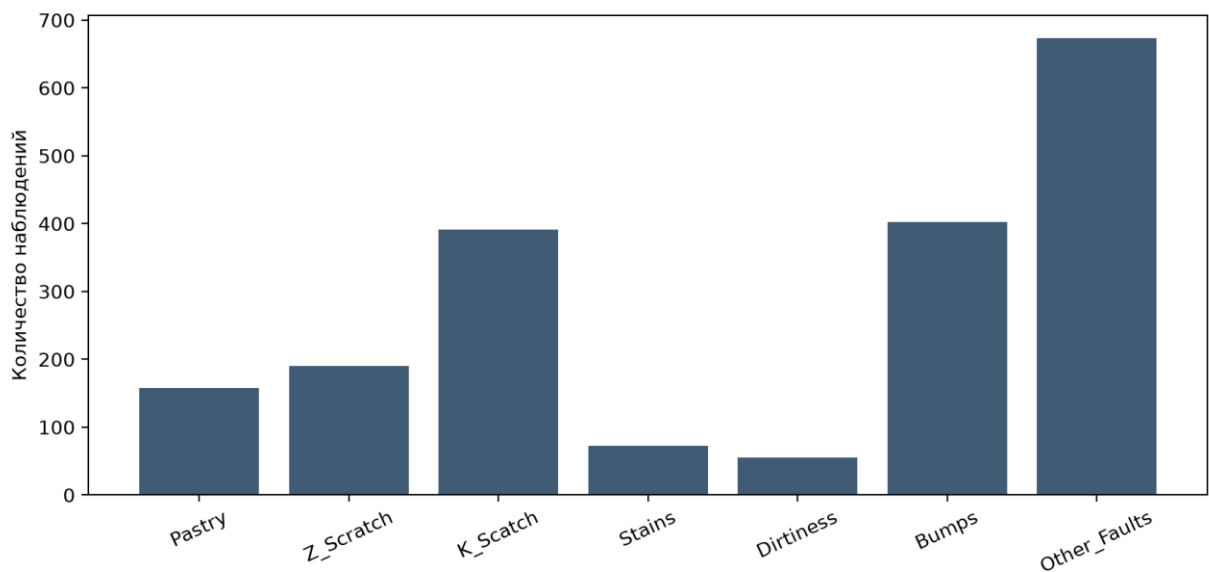


Рисунок 1 – Распределение исходных классов

Класс Other_Faults содержит 673 строки, а Dirtiness — 55. Поэтому основным критерием выбора принята macro-F1, которая придаёт каждому классу одинаковый вес. Стратифицированное разбиение сохраняет присутствие всех семи классов и в обучающей, и в тестовой выборке.

3 Признаки и подготовка выборки

Все 27 входных столбцов преобразованы в числовой тип float. Наименования ниже сгруппированы по смыслу имени для чтения; сами значения не заменялись. Подробные физические единицы большинства полей не указаны в карточке UCI, поэтому они не приписываются данным.

Сведения приведены в таблице 3.

Таблица 3 – Полный перечень входных признаков

Группа	Имена столбцов	Число
Геометрические поля	X_Minimum, X_Maximum, Y_Minimum, Y_Maximum, Pixels_Areas, X_Perimeter, Y_Perimeter	7
Яркость	Sum_of_Luminosity, Minimum_of_Luminosity, Maximum_of_Luminosity	3
Параметры стали и линии	Length_of_Conveyer, TypeOfSteel_A300, TypeOfSteel_A400, Steel_Plate_Thickness	4
Производные индексы	Edges_Index, Empty_Index, Square_Index, Outside_X_Index, Edges_X_Index, Edges_Y_Index, Outside_Global_Index, LogOfAreas, Log_X_Index, Log_Y_Index, Orientation_Index, Luminosity_Index, SigmoidOfAreas	13

Масштабирование и заполнение пропусков не применялись: признаки конечны, пропусков нет, а обе выбранные модели основаны на деревьях. Целевая переменная получена как индекс единственного активного целевого индикатора. Значения семи индикаторов не могут попасть в матрицу входа.

Функция `train_test_split` с `test_size=0,2`, `stratify=y` и `random_state=42` сформировала 1 552 строки для обучения и 389 для теста. Индексы обеих частей сохранены. Проверены отсутствие общих индексов, полное покрытие 1 941 строки и отсутствие идентичных строк признаков по разные стороны разбиения.

Параметры подбирались только на обучающей части. `StratifiedKFold` с тремя фолдами, `shuffle=True` и `random_state=42` был сформирован один раз и использован для обоих алгоритмов. Отложенный тест не участвовал в подборе, ранней остановке или преобразовании признаков.

4 Реализация моделей и подбор параметров

BaggingClassifier объединяет 80 независимо обучаемых DecisionTreeClassifier. Bootstrap-выборка содержит 1 241 извлечение с возвращением: 80% train с округлением вниз; уникальных строк меньше. Все признаки доступны каждому дереву, случайного отбора признаков нет. Класс определяется по усреднённым вероятностям [3].

Бустинг реализован GradientBoostingClassifier с log_loss, learning_rate=0,1 и subsample=1. На каждой итерации деревья последовательно приближают отрицательный градиент потерь; для семи классов строятся семь регрессионных деревьев на итерацию [4]. Выбраны 160 итераций и глубина 3, то есть 1 120 деревьев, а не 160 отдельных деревьев.

Каждая сетка содержит четыре комбинации параметров; «лист» означает минимальное число наблюдений в листе. GridSearchCV выбирает максимум macro-F1 на трёх общих фолдах: по 12 обучений на алгоритм. n_jobs=1 и refit=False; итоговые модели обучаются отдельно.

Сведения приведены в таблице 4.

Таблица 4 – Результаты всех комбинаций на трёх CV-фолдах

Модель	Глубина	Настройка	Macro-F1	σ
Bagging	6	Лист: 2	0,7163	0,0096
Bagging	6	Лист: 5	0,7139	0,0052
Bagging	Нет предела	Лист: 2	0,7669	0,0125
Bagging	Нет предела	Лист: 5	0,7526	0,0196
Boosting	2	Итерации: 80	0,7690	0,0046
Boosting	2	Итерации: 160	0,7775	0,0055
Boosting	3	Итерации: 80	0,7759	0,0063
Boosting	3	Итерации: 160	0,7813	0,0106

σ — стандартное отклонение трёх CV-оценок, не доверительный интервал. Максимум: бэггинг без предела глубины, лист 2; бустинг — 160 итераций, глубина 3. Время поиска: 4,036 / 24,790 с; итоговый fit измеряется отдельно.

5 Среда и измерение времени

Сведения приведены в таблице 5.

Таблица 5 – Фактическая среда запуска

Параметр	Значение
Место исполнения	local Windows verification
Операционная система	Windows-11-10.0.26200-SP0
Python / scikit-learn	3.12.14 / 1.9.1
NumPy / Pandas	2.3.5 / 3.0.1
Логические CPU / вычислительные потоки	16 / 1
Ускоритель	CPU; GPU не используется
Завершение запуска	05.10.2026 13:04:58 UTC+3

Для каждой выбранной конфигурации выполнены три новых обучения. Порядок чередовался: Bagging–Boosting, Boosting–Bagging, Bagging–Boosting. `time.perf_counter` измеряет прошедшее время. Fit не включает загрузку, CV-поиск и построение графиков; predict включает предсказание всех 389 тестовых строк.

Сведения приведены в таблице 6.

Таблица 6 – Все исходные замеры итоговых моделей

Модель	Повтор	Fit, c	Predict, c
Bagging	1	0,591103	0,006413
Bagging	2	0,586019	0,006566
Bagging	3	0,589319	0,006198
Boosting	1	4,714289	0,002888
Boosting	2	4,715257	0,002923
Boosting	3	4,717341	0,002923

Сравнивается медиана трёх замеров. `n_jobs=1` и `threadpool_limits(limits=1)` ограничивают параллелизм; состояние пулов сохранено внутри этого контекста. Фиксированный `random_state` дал одинаковые предсказания. Повторы оценивают время, но не независимое качество. CPU-замеры этой реализации не сопоставляются с GPU-минутами примера методички.

6 Сравнение качества и времени

Ассигасу — доля правильно классифицированных наблюдений. Balanced ассигасу — средняя полнота по семи классам. F1 каждого класса — гармоническое среднее точности и полноты; макро-F1 — среднее семи таких F1 без весов по частоте. Для дополнительной проверки использованы постоянный прогноз наиболее частого класса и одно дерево глубины 8 с минимумом 2 строки в листе.

Сведения приведены в таблице 7.

Таблица 7 – Качество на общем отложенном тесте

Модель	Accuracy	Balanced accuracy	Macro-F1 test	Macro-F1 train
Most frequent	0,3470	—	0,0736	—
One tree	0,7172	—	0,7442	—
Bagging	0,7918	0,8041	0,8119	0,9791
Boosting	0,8098	0,8218	0,8268	0,9975

На рисунке 2 сопоставлены test macro-F1 и медиана времени итогового fit. Время подбора параметров сюда не входит.

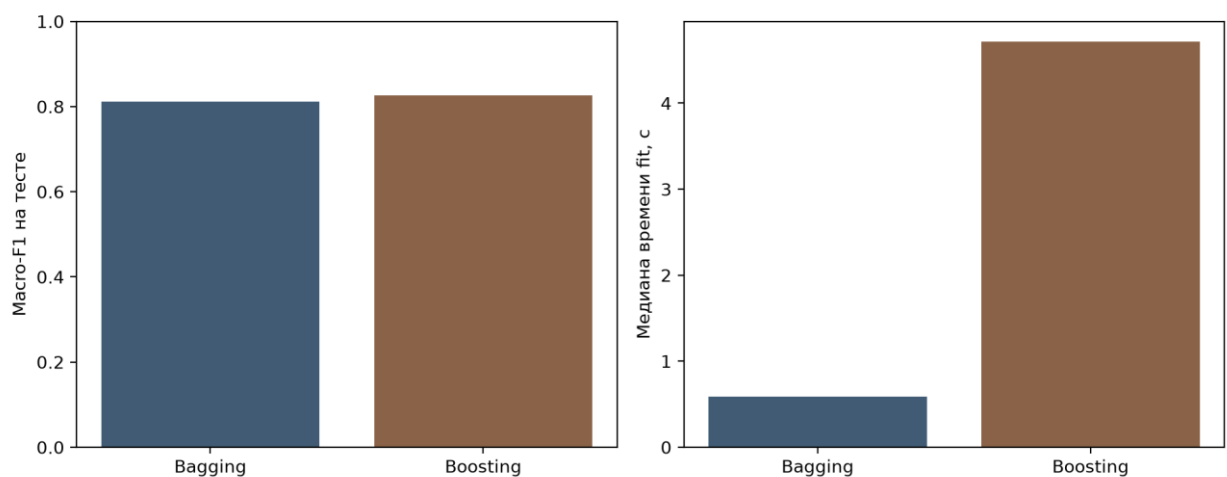


Рисунок 2 – Сопоставление макро-F1 и времени обучения

Бустинг улучшил макро-F1 на 0,0149 и ассигасу на 0,0180; это разности долей, а не относительные проценты. Обучение бустинга заняло примерно в 8,00 раза больше времени. Медиана predict составила 0,006413 с у бэггинга и 0,002923 с у бустинга; на столь коротких замерах нельзя делать универсальный вывод о скорости обслуживания.

7 Ошибки по классам

Сведения приведены в таблице 8.

Таблица 8 – F1 и число тестовых наблюдений каждого класса

Класс	Test	F1 Bagging	F1 Boosting
Pastry	32	0,6780	0,6230
Z_Scratch	38	0,8800	0,9250
K_Scratch	78	0,9487	0,9740
Stains	14	0,9630	0,9630
Dirtiness	11	0,7826	0,8182
Bumps	81	0,6795	0,7273
Other_Faults	135	0,7518	0,7571

На рисунке 3 строки матрицы обозначают истинные классы, столбцы — предсказанные. Все семь классов показаны в порядке исходного словаря.

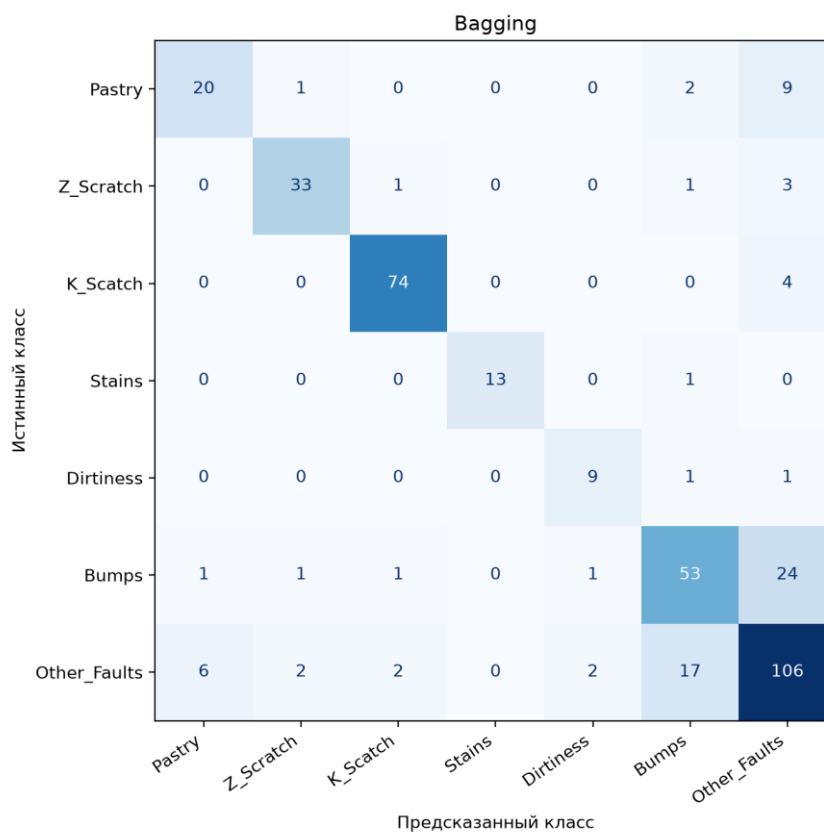


Рисунок 3 – Матрица ошибок бэггинга

Бэггинг отнёс 24 из 81 наблюдения Bumps к Other_Faults. Для Pastry правильно распознаны 20 из 32 наблюдений. Матрица сохраняет конкретные числа ошибок, которые не видны в одной общей accuracy.

8 Матрица ошибок бустинга и ограничения

На рисунке 4 приведена матрица бустинга на тех же 389 строках. Порядок классов и шкала счёта совпадают с рисунком 3.

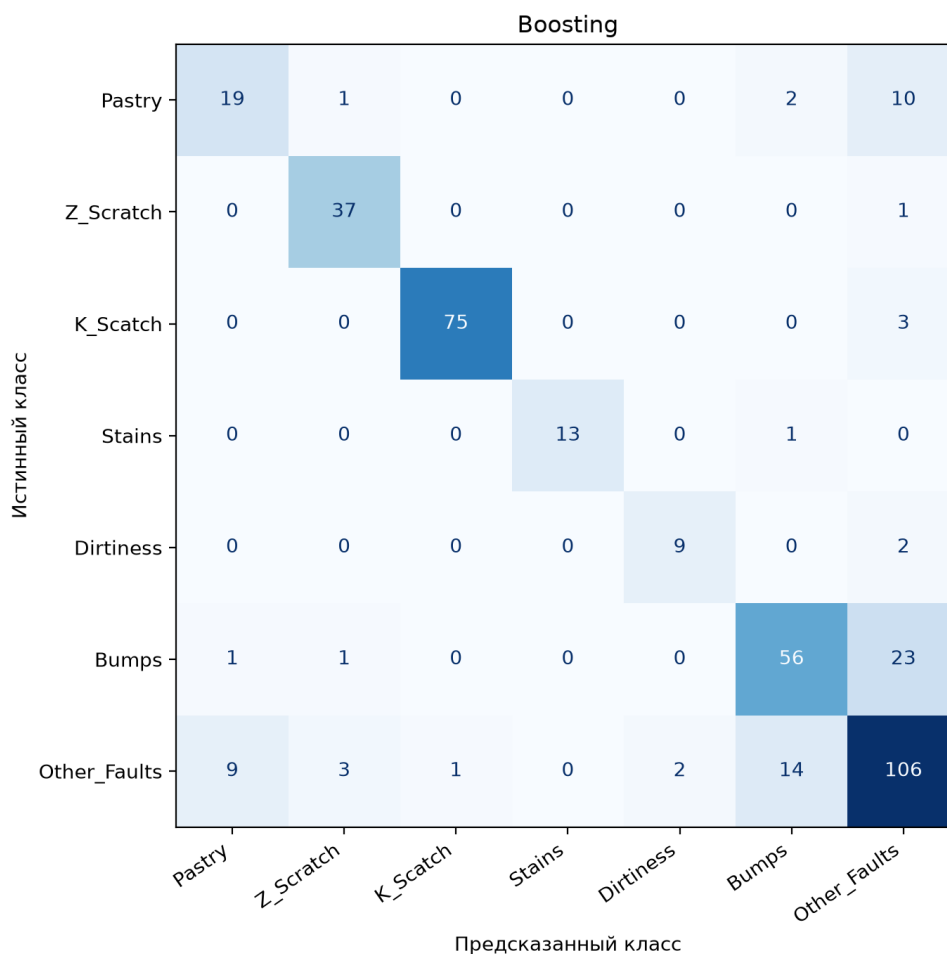


Рисунок 4 – Матрица ошибок градиентного бустинга

У бустинга 23 ошибки Bumps→Other_Faults. При лучшем общем макро-F1 результат Pastry снизился: 19 правильных ответов вместо 20, F1 0,6230 вместо 0,6780. Для Stains F1 одинаков. Поэтому среднее улучшение не означает улучшения каждого класса.

На train макро-F1 равна 0,9791 и 0,9975; заметный разрыв с test указывает на риск переобучения. В тесте Dirtiness лишь 11 строк, Stains — 14, поэтому оценка редких классов чувствительна к отдельным ошибкам. Один holdout и небольшие сетки не доказывают статистически значимого или универсального превосходства; отдельного внешнего набора для проверки переноса здесь нет.

9 Проверка и воспроизведение

Семь ячеек notebook выполнены локально без ошибок. `verify_experiment.py` прошёл 45 проверок, независимо переобучив обе модели и воспроизведя все предсказания. SHA-256 связывает отчёт с результатами.

Сведения приведены в таблице 9.

Таблица 9 – Артефакты воспроизводимого эксперимента

Файл или каталог	Содержимое
Блокнот IPYNB	Код, пояснения и сохранённые outputs всех ячеек
data/steel_plates_faults.zip	Неизменённый исходный архив UCI
outputs/results.json	Среда, индексы, параметры, метрики и замеры
outputs/cv_*.csv	Все четыре комбинации каждой модели
outputs/predictions_*.csv	389 истинных и предсказанных меток обеих моделей
outputs/verification.json	Независимые проверки и SHA-256 артефактов

Исполнение в Google Colab

1. В Colab выбрать Файл → Загрузить блокнот и открыть приложенный IPYNB. Данные автоматически загрузятся с UCI.

2. Выбрать CPU и выполнить все ячейки. Первая ячейка устанавливает версии библиотек; Execution environment должен содержать Google Colab.

3. Дождаться ALL CELLS COMPLETED и скачать выполненный IPYNB и outputs. Для облачного отчёта использовать новые замеры этого запуска.

Облачный запуск и уникальность набора в группе не подтверждены. Перед сдачей необходимо выполнить notebook в Colab и согласовать данные с группой. Локальные результаты не заменяют эти условия.

Вывод

На данном тесте бустинг точнее, бэггинг быстрее обучается; оба лучше выбранных базовых моделей. Ограничения — ухудшение Pastry, малочисленные редкие классы и риск переобучения. Индексы, параметры и предсказания сохранены для проверки выводов.

Список использованных источников

1. Практическая работа №7. Ансамблевое обучение. Предоставленные методические материалы, 12 с. Практическое задание на с. 12, требование Colab Notebook на с. 11.
2. Buscema M., Terzi S., Tastle W. Steel Plates Faults. UCI Machine Learning Repository, 2010. DOI: <https://doi.org/10.24432/C5J88N>. Лицензия CC BY 4.0. Дата обращения: 05.10.2026.
3. scikit-learn developers. BaggingClassifier. Документация 1.9.1. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>. Дата обращения: 05.10.2026.
4. scikit-learn developers. GradientBoostingClassifier. Документация 1.9.1. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>. Дата обращения: 05.10.2026.